



# Recursion in Java

## PLACEMENT PREPARATION NOTES

Concepts · Call Stack Tracing · 16 Must-Know Interview Questions

### Contents

#### Core Concepts

1. What is Recursion?
2. How Recursion Actually Works (Calling Phase)
3. The Returning Phase — Ascending vs Descending Order
4. Recursion vs Loop
5. The Call Stack in Recursion

#### Frequently Asked Recursion Questions

1. Print 1 to N
2. Print N to 1
3. Sum of First N Numbers
4. Factorial
5. Power of a Number
6. Fibonacci Series
7. Reverse a String
8. Palindrome Check
9. Sum of Digits
10. Count Digits
11. Reverse a Number
12. GCD Using Recursion
13. Recursive Binary Search
14. Recursive Array Problems — Check Sorted / Find Max

## 1 What is Recursion?

Recursion means a method calling itself to solve a smaller version of the same problem.

#### Example

```
void fun(int n) {  
    if (n == 0)  
        return;  
  
    System.out.println(n);  
    fun(n - 1);  
}
```

If we call:

```
fun(5);
```

#### OUTPUT

```
5  
4  
3  
2  
1
```

#### Every recursion must have 2 things

##### ① Base Case

Condition that stops recursion.

```
if (n == 0)  
    return;
```

##### ② Recursive Case

The method calls itself with a smaller / simpler input.

```
fun(n - 1);
```

## 2 How Recursion Actually Works

Consider:

```
void fun(int n) {
    if (n == 0)
        return;

    System.out.println(n);
    fun(n - 1);
}
```

For `fun(3);` — think of it as the **calling phase**:

```
fun(3)
↓
print 3
↓
fun(2)
↓
print 2
↓
fun(1)
↓
print 1
↓
fun(0)
↓
return
```

## 3 The Returning Phase — A Common Interview Trap

This is where many placement questions come from. Look at this variation:

```
void fun(int n) {
    if (n == 0)
        return;

    fun(n - 1);
    System.out.println(n);
}
```

Call: `fun(3);`

### OUTPUT

```
1
2
3
```

**Why?** The calls happen first — this is the calling phase:

```
fun(3)
↓
fun(2)
↓
fun(1)
↓
fun(0)
```

Then the methods return one by one, printing on the way back:

```
print 1
print 2
print 3
```

### Remember this rule:

`System.out.println(n);` then `fun(n - 1);` → Printing happens **before** recursion → **descending** order.

`fun(n - 1);` then `System.out.println(n);` → Printing happens **after** recursion → **ascending** order.

This is a very common interview / output-prediction question.

## 4 Recursion vs Loop

Example — print 1 to N.

### Using Loop

```
for (int i = 1; i <= n; i++) {  
    System.out.println(i);  
}
```

### Using Recursion

```
void print(int n) {  
    if (n == 0)  
        return;  
  
    print(n - 1);  
    System.out.println(n);  
}
```

Both solve the problem, but recursion uses the **call stack** instead of an explicit loop counter.

## 5 Stack in Recursion

For `fun(3)`, the stack looks approximately like:

```
fun(3)  
fun(2)  
fun(1)  
fun(0)
```

After reaching the base case, functions return one by one, popping off the stack.

**Key point:** This is why recursion requires  **$O(n)$  stack space** in many simple recursive problems.

## ★ Frequently Asked Recursion Questions

For placements, practice these in the order given below.

### LEVEL 1 — MUST KNOW

#### 1 Print 1 to N

```
void print(int n) {  
    if (n == 0)  
        return;  
  
    print(n - 1);  
    System.out.println(n);  
}
```

**TC:  $O(n)$**    **SC:  $O(n)$**

#### 2 Print N to 1

```
void print(int n) {  
    if (n == 0)  
        return;  
  
    System.out.println(n);  
    print(n - 1);  
}
```

#### 3 Sum of First N Numbers

Problem: `1 + 2 + 3 + ... + N`

```
int sum(int n) {  
    if (n == 0)  
        return 0;  
  
    return n + sum(n - 1);  
}
```

Example: `sum(5)`

```
5 + sum(4)
5 + 4 + sum(3)
5 + 4 + 3 + sum(2)
5 + 4 + 3 + 2 + sum(1)
5 + 4 + 3 + 2 + 1
= 15
```

**TC:  $O(n)$**  **SC:  $O(n)$**

#### 4 Factorial ★★★ (Very frequently asked)

$5! = 5 \times 4 \times 3 \times 2 \times 1$

```
int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;

    return n * factorial(n - 1);
}
```

Calling `factorial(5)` works like:

```
5 × factorial(4)
  ↓
5 × 4 × factorial(3)
  ↓
5 × 4 × 3 × factorial(2)
  ↓
5 × 4 × 3 × 2 × factorial(1)
  ↓
120
```

**TC:  $O(n)$**  **SC:  $O(n)$**

#### 5 Power of a Number ★★★

Calculate:  $2^5 = 32$

```
int power(int a, int n) {
    if (n == 0)
        return 1;

    return a * power(a, n - 1);
}
```

Example: `power(2, 5)`

**TC:  $O(n)$**  **SC:  $O(n)$**

**Interview follow-up:** "Can you optimize this?" — Yes, using **fast exponentiation**,  $a^n$  can be calculated in  $O(\log n)$ . This is a slightly higher-level recursion question.

#### 6 Fibonacci ★★★ (One of the most famous recursion questions)

Sequence: 0 1 1 2 3 5 8 13 ...

Recursive definition:  $F(n) = F(n-1) + F(n-2)$

```
int fib(int n) {
    if (n <= 1)
        return n;

    return fib(n - 1) + fib(n - 2);
}
```

Example: `fib(5)` → `fib(4) + fib(3)`, and each of those branches again creates two calls.

**Naive TC:  $O(2^n)$**  **SC:  $O(n)$**

⚠ **Important interview point** — "Is this an efficient solution?"

**Answer:** No. The recursive solution recalculates the same Fibonacci values many times. We can optimize it using Dynamic Programming / memoization or an iterative approach.

#### 7 Reverse a String ★★★

Example: `hello` → `olleh`

```
void reverse(String s, int index) {
    if (index < 0)
        return;

    System.out.print(s.charAt(index));
    reverse(s, index - 1);
}
```

Call: `reverse("hello", 4);`

#### OUTPUT

olleh

### 8 Palindrome Check ★★★

Example: `madam → palindrome`, `hello → not palindrome`

```
boolean isPalindrome(String s, int left, int right) {

    if (left >= right)
        return true;

    if (s.charAt(left) != s.charAt(right))
        return false;

    return isPalindrome(s, left + 1, right - 1);
}
```

Call: `isPalindrome("madam", 0, 4);`

```
m a d a m
  ↑      ↑
compare

a d a
  ↑      ↑
compare

d
```

TC: O(n)

SC: O(n)

### 9 Sum of Digits ★★★

Example: `1234 → 1 + 2 + 3 + 4 = 10`

```
int sumDigits(int n) {

    if (n == 0)
        return 0;

    return (n % 10) + sumDigits(n / 10);
}
```

For 1234:

```
4 + sumDigits(123)
4 + 3 + sumDigits(12)
4 + 3 + 2 + sumDigits(1)
4 + 3 + 2 + 1
= 10
```

### 10 Count Digits ★★

```
int countDigits(int n) {

    if (n == 0)
        return 0;

    return 1 + countDigits(n / 10);
}
```

For `12345` → Answer: `5`

### 11 Reverse a Number ★★★

Example: `1234 → 4321`

One common approach:

```
int reverse(int n, int result) {

    if (n == 0)
        return result;

    return reverse(n / 10, result * 10 + n % 10);
}
```

Call: `reverse(1234, 0);`

## 12 GCD Using Recursion ★★★ (Very good placement question)

Example: `GCD(12, 18) = 6`

Euclidean algorithm:

```
int gcd(int a, int b) {

    if (b == 0)
        return a;

    return gcd(b, a % b);
}
```

For `gcd(18, 12)` :

```
gcd(18,12)
gcd(12,6)
gcd(6,0)
6
```

**TC:  $O(\log(\min(a,b)))$  approx.**

## 13 Recursive Binary Search ★★★ (Combines recursion + searching)

```
int binarySearch(int[] arr, int low, int high, int target) {

    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == target)
        return mid;

    if (target < arr[mid])
        return binarySearch(arr, low, mid - 1, target);

    return binarySearch(arr, mid + 1, high, target);
}
```

**TC:  $O(\log n)$**

**SC:  $O(\log n)$**

## 14 Recursive Array Problems ★★★

**Check if array is sorted**

```
boolean sorted(int[] arr, int i) {

    if (i == arr.length - 1)
        return true;

    if (arr[i] > arr[i + 1])
        return false;

    return sorted(arr, i + 1);
}
```

**Find maximum element**

```
int max(int[] arr, int i) {

    if (i == arr.length - 1)
        return arr[i];

    return Math.max(arr[i], max(arr, i + 1));
}
```

| #  | Problem                          | Time Complexity      | Space Complexity     |
|----|----------------------------------|----------------------|----------------------|
| 1  | Print 1 to N                     | $O(n)$               | $O(n)$               |
| 2  | Print N to 1                     | $O(n)$               | $O(n)$               |
| 3  | Sum of first N numbers           | $O(n)$               | $O(n)$               |
| 4  | Factorial                        | $O(n)$               | $O(n)$               |
| 5  | Power of a number (naive)        | $O(n)$               | $O(n)$               |
| 6  | Fibonacci (naive)                | $O(2^n)$             | $O(n)$               |
| 7  | Reverse a string                 | $O(n)$               | $O(n)$               |
| 8  | Palindrome check                 | $O(n)$               | $O(n)$               |
| 9  | Sum of digits                    | $O(\log n)$          | $O(\log n)$          |
| 10 | Count digits                     | $O(\log n)$          | $O(\log n)$          |
| 11 | Reverse a number                 | $O(\log n)$          | $O(\log n)$          |
| 12 | GCD (Euclidean)                  | $O(\log(\min(a,b)))$ | $O(\log(\min(a,b)))$ |
| 13 | Recursive binary search          | $O(\log n)$          | $O(\log n)$          |
| 14 | Array sorted check / max element | $O(n)$               | $O(n)$               |